

DATA STRUCTURES – DAY 4

PLACEMENT PREPARATION GUIDE

Queues (ADT, Circular, Deque, Priority) • Recursion & Backtracking
Hashing (Tables, Collisions, Applications)
Trees & BST • Heaps • AVL & Red-Black Trees

Concept Explanations + Two Java Programs for Every Topic

Table of Contents

1. Queue ADT and Basic Operations
2. Circular Queue
3. Deque and Priority Queue
4. Queue Implementation Using Array and Linked List
5. Recursion Fundamentals and Recursive Algorithms
6. Backtracking Using Recursion
7. Hash Tables and Hash Functions
8. Collision Resolution: Chaining and Open Addressing
9. Applications of Hashing and Hash-Based Problems
10. Tree Terminology and Binary Trees
11. Binary Tree Traversals: Preorder, Inorder, Postorder
12. Level Order Traversal and Height of Binary Tree
13. BST Operations: Search, Insert and Delete
14. BST Applications and Validation Problems
15. Min Heap and Max Heap
16. Heap Sort and Heapify
17. Heap Operations and Priority Queue Applications
18. AVL Trees and Rotations
19. Red-Black Trees and Self-Balancing Concepts

1. Queue ADT and Basic Operations

Queue = **FIFO** (First-In-First-Out). Abstract operations: **enqueue(x)** (insert at rear), **dequeue()** (remove from front), **front/peek()**, **isEmpty()**, **size()**. All ideally $O(1)$.

Example 1: Using java.util.Queue

```
import java.util.LinkedList;
import java.util.Queue;
public class QueueADT1 {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();
        q.offer(10); q.offer(20); q.offer(30); // enqueue
        System.out.println(q.peek());        // 10
        System.out.println(q.poll());         // 10 (dequeue)
        System.out.println(q.poll());        // 20
        System.out.println(q.isEmpty());      // false
    }
}
```

Example 2: Simple application – generate binary numbers 1..N

```
import java.util.*;
public class QueueADT2 {
    static void generateBinary(int n) {
        Queue<String> q = new LinkedList<>();
        q.offer("1");
        for (int i = 0; i < n; i++) {
            String s = q.poll();
            System.out.print(s + " ");
            q.offer(s + "0");
            q.offer(s + "1");
        }
    }
    public static void main(String[] args) { generateBinary(5); }
    // 1 10 11 100 101
}
```

2. Circular Queue

In a linear array queue, space after dequeue is wasted. Circular queue reuses space by wrapping indices with modulo: $(\text{index} + 1) \% \text{capacity}$. Full condition and empty condition must be distinguished carefully (often keep a count or reserve one slot).

Example 1: Circular Queue implementation

```
class CircularQueue {
    int[] a; int front, rear, size, cap;
    CircularQueue(int c) { cap = c; a = new int[c]; front = 0; rear = -1; size = 0; }
    boolean isEmpty() { return size == 0; }
    boolean isFull() { return size == cap; }
    void enqueue(int x) {
        if (isFull()) throw new RuntimeException("Full");
        rear = (rear + 1) % cap;
        a[rear] = x; size++;
    }
    int dequeue() {
        if (isEmpty()) throw new RuntimeException("Empty");
        int v = a[front];
        front = (front + 1) % cap; size--;
        return v;
    }
}

public class CircularQ1 {
    public static void main(String[] args) {
        CircularQueue q = new CircularQueue(3);
        q.enqueue(1); q.enqueue(2); q.enqueue(3);
        System.out.println(q.dequeue()); // 1
        q.enqueue(4);
        System.out.println(q.dequeue()); // 2
    }
}
```

Example 2: Demonstrate wrap-around

```
public class CircularQ2 {
    public static void main(String[] args) {
        CircularQueue q = new CircularQueue(4);
        q.enqueue(10); q.enqueue(20); q.enqueue(30);
        q.dequeue(); q.dequeue(); // free two slots
        q.enqueue(40); q.enqueue(50); // wrap around
        while (!q.isEmpty()) System.out.print(q.dequeue() + " ");
        // 30 40 50
    }
}
```

3. Deque and Priority Queue

Deque (Double-Ended Queue): insert/delete at both ends. Java: `ArrayDeque`, `LinkedList`.

Priority Queue: elements ordered by priority (heap-based). Java: `PriorityQueue` (min-heap by default).

Example 1: Deque operations

```
import java.util.ArrayDeque;
import java.util.Deque;
public class DequeDemo {
    public static void main(String[] args) {
        Deque<Integer> dq = new ArrayDeque<>();
        dq.addFirst(10); dq.addLast(20); dq.addFirst(5);
        System.out.println(dq);           // [5, 10, 20]
        System.out.println(dq.removeLast()); // 20
        System.out.println(dq.removeFirst()); // 5
    }
}
```

Example 2: PriorityQueue (min-heap)

```
import java.util.PriorityQueue;
public class PQDemo {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap
        pq.offer(30); pq.offer(10); pq.offer(20);
        while (!pq.isEmpty()) System.out.print(pq.poll() + " "); // 10 20 30
        // Max-heap: new PriorityQueue<>(Collections.reverseOrder())
    }
}
```

4. Queue Implementation Using Array and Linked List

Array: fixed or circular. Linked list: enqueue at tail, dequeue at head (keep both head & tail pointers for $O(1)$).

Example 1: Linked-list Queue

```
class QNode { int data; QNode next; QNode(int d){data=d;} }
class LLQueue {
    QNode front, rear;
    void enqueue(int x) {
        QNode n = new QNode(x);
        if (rear == null) { front = rear = n; return; }
        rear.next = n; rear = n;
    }
    int dequeue() {
        if (front == null) throw new RuntimeException("Empty");
        int v = front.data; front = front.next;
        if (front == null) rear = null;
        return v;
    }
    boolean isEmpty() { return front == null; }
}
public class QueueImpl1 {
    public static void main(String[] args) {
        LLQueue q = new LLQueue();
        q.enqueue(1); q.enqueue(2); q.enqueue(3);
        System.out.println(q.dequeue()); // 1
    }
}
```

Example 2: Simple linear array queue (with front/rear)

```
class ArrayQueue {
    int[] a; int front = 0, rear = -1, size = 0;
    ArrayQueue(int cap) { a = new int[cap]; }
    void enqueue(int x) {
        if (size == a.length) throw new RuntimeException("Full");
        a[++rear] = x; size++;
    }
    int dequeue() {
        if (size == 0) throw new RuntimeException("Empty");
        int v = a[front++]; size--;
        return v;
    }
}
public class QueueImpl2 {
    public static void main(String[] args) {
        ArrayQueue q = new ArrayQueue(5);
        q.enqueue(10); q.enqueue(20);
        System.out.println(q.dequeue()); // 10
    }
}
```

5. Recursion Fundamentals and Recursive Algorithms

A function calling itself. Needs a **base case** (termination) and a **recursive case**. Each call adds a frame to the call stack. Classic examples: factorial, Fibonacci, tower of Hanoi, tree traversals, binary search.

Example 1: Fibonacci (naive + note on complexity)

```
public class Recursion1 {
    static int fib(int n) {
        if (n <= 1) return n;           // base
        return fib(n - 1) + fib(n - 2); // O(2^n)
    }
    public static void main(String[] args) {
        System.out.println(fib(6)); // 8
    }
}
```

Example 2: Tower of Hanoi

```
public class Recursion2 {
    static void hanoi(int n, char from, char to, char aux) {
        if (n == 0) return;
        hanoi(n - 1, from, aux, to);
        System.out.println("Move disk " + n + " from " + from + " to " + to);
        hanoi(n - 1, aux, to, from);
    }
    public static void main(String[] args) {
        hanoi(3, 'A', 'C', 'B');
    }
}
```

6. Backtracking Using Recursion

Backtracking = explore all possibilities, abandon a path as soon as it cannot lead to a solution ("undo" the choice). Used for: N-Queens, Sudoku, permutations, subset-sum, maze paths, etc.

Example 1: Generate all subsets (power set)

```
import java.util.*;
public class Backtrack1 {
    static void subsets(int[] a, int i, List<Integer> cur, List<List<Integer>> res) {
        if (i == a.length) { res.add(new ArrayList<>(cur)); return; }
        cur.add(a[i]); subsets(a, i + 1, cur, res); // include
        cur.remove(cur.size() - 1);               // backtrack
        subsets(a, i + 1, cur, res);               // exclude
    }
    public static void main(String[] args) {
        List<List<Integer>> res = new ArrayList<>();
        subsets(new int[]{1, 2}, 0, new ArrayList<>(), res);
        System.out.println(res); // [[1,2],[1],[2],[]]
    }
}
```

Example 2: Print all permutations of a string

```
public class Backtrack2 {
    static void permute(char[] a, int l) {
        if (l == a.length) { System.out.println(new String(a)); return; }
        for (int i = l; i < a.length; i++) {
            swap(a, l, i);
            permute(a, l + 1);
            swap(a, l, i); // backtrack
        }
    }
    static void swap(char[] a, int i, int j) {
        char t = a[i]; a[i] = a[j]; a[j] = t;
    }
    public static void main(String[] args) {
        permute("AB".toCharArray(), 0); // AB BA
    }
}
```

7. Hash Tables and Hash Functions

Hash table = array + hash function that maps a key to an index. Ideal average $O(1)$ insert/search/delete. Hash function goals: uniform distribution, fast computation, determinism. Java: HashMap, HashSet, hashCode().

Example 1: Simple hash function & table idea

```
public class Hash1 {
    static int hash(String key, int m) {
        int h = 0;
        for (char c : key.toCharArray()) h = (h * 31 + c) % m;
        return Math.abs(h);
    }
    public static void main(String[] args) {
        System.out.println(hash("apple", 10));
        System.out.println(hash("banana", 10));
    }
}
```

Example 2: Frequency count with HashMap

```
import java.util.*;
public class Hash2 {
    public static void main(String[] args) {
        String s = "programming";
        Map<Character, Integer> freq = new HashMap<>();
        for (char c : s.toCharArray())
            freq.put(c, freq.getOrDefault(c, 0) + 1);
        System.out.println(freq);
    }
}
```

8. Collision Resolution: Chaining and Open Addressing

Chaining: each slot holds a linked list (or tree) of colliding keys. Easy to implement; load factor can be > 1 .

Open Addressing: probe for another empty slot (linear, quadratic, double hashing). Load factor must stay < 1 . Clustering can degrade performance.

Example 1: Chaining (array of lists)

```
import java.util.*;
public class Collision1 {
    static final int M = 7;
    static List<Integer>[] table = new List[M];
    static {
        for (int i = 0; i < M; i++) table[i] = new LinkedList<>();
    }
    static void insert(int key) {
        int idx = key % M;
        table[idx].add(key);
    }
    static boolean search(int key) {
        return table[key % M].contains(key);
    }
    public static void main(String[] args) {
        insert(10); insert(17); insert(24); // all hash to 3
        System.out.println(search(17)); // true
        System.out.println(table[3]);    // [10, 17, 24]
    }
}
```

Example 2: Linear probing (open addressing sketch)

```
public class Collision2 {
    static Integer[] table = new Integer[7];
    static int hash(int k) { return k % 7; }
    static void insert(int key) {
        int i = hash(key);
        while (table[i] != null) i = (i + 1) % 7; // linear probe
        table[i] = key;
    }
    public static void main(String[] args) {
        insert(10); insert(17); insert(3);
        for (Integer x : table) System.out.print(x + " ");
    }
}
```

9. Applications of Hashing and Hash-Based Problems

Applications: dictionaries, sets, caching, databases, compilers (symbol tables), anagrams, two-sum, subarray sum, first unique character, counting sort variants, etc.

Example 1: Two-sum using HashMap

```
import java.util.*;
public class HashApp1 {
    static int[] twoSum(int[] a, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < a.length; i++) {
            int need = target - a[i];
            if (map.containsKey(need)) return new int[]{map.get(need), i};
            map.put(a[i], i);
        }
        return null;
    }
    public static void main(String[] args) {
        int[] res = twoSum(new int[]{2, 7, 11, 15}, 9);
        System.out.println(res[0] + " " + res[1]); // 0 1
    }
}
```

Example 2: Check if array has duplicates (HashSet)

```
import java.util.*;
public class HashApp2 {
    static boolean hasDuplicate(int[] a) {
        Set<Integer> set = new HashSet<>();
        for (int x : a) if (!set.add(x)) return true;
        return false;
    }
    public static void main(String[] args) {
        System.out.println(hasDuplicate(new int[]{1, 2, 3, 1})); // true
    }
}
```

10. Tree Terminology and Binary Trees

Tree: hierarchical, acyclic, connected. Terms: root, parent, child, leaf, sibling, ancestor, descendant, depth, height, level, subtree, degree. Binary tree: each node has at most 2 children (left, right).

Example 1: Binary Tree node & simple build

```
class TNode {
    int data; TNode left, right;
    TNode(int d) { data = d; }
}

public class Tree1 {
    public static void main(String[] args) {
        TNode root = new TNode(1);
        root.left = new TNode(2);
        root.right = new TNode(3);
        root.left.left = new TNode(4);
        //      1
        //     / \
        //    2  3
        //   /
        //  4
        System.out.println(root.left.left.data); // 4
    }
}
```

Example 2: Count nodes / leaves (recursive)

```
public class Tree2 {
    static int countNodes(TNode r) {
        if (r == null) return 0;
        return 1 + countNodes(r.left) + countNodes(r.right);
    }
    static int countLeaves(TNode r) {
        if (r == null) return 0;
        if (r.left == null && r.right == null) return 1;
        return countLeaves(r.left) + countLeaves(r.right);
    }
    public static void main(String[] args) {
        TNode root = new TNode(1);
        root.left = new TNode(2); root.right = new TNode(3);
        root.left.left = new TNode(4);
        System.out.println(countNodes(root)); // 4
        System.out.println(countLeaves(root)); // 2
    }
}
```

11. Binary Tree Traversals: Preorder, Inorder and Postorder

- **Preorder:** Root → Left → Right
- **Inorder:** Left → Root → Right (gives sorted order for BST)
- **Postorder:** Left → Right → Root

Example 1: All three recursive traversals

```
public class Traversal1 {
    static void preorder(TNode r) {
        if (r == null) return;
        System.out.print(r.data + " ");
        preorder(r.left); preorder(r.right);
    }
    static void inorder(TNode r) {
        if (r == null) return;
        inorder(r.left);
        System.out.print(r.data + " ");
        inorder(r.right);
    }
    static void postorder(TNode r) {
        if (r == null) return;
        postorder(r.left); postorder(r.right);
        System.out.print(r.data + " ");
    }
    public static void main(String[] args) {
        TNode root = new TNode(1);
        root.left = new TNode(2); root.right = new TNode(3);
        root.left.left = new TNode(4); root.left.right = new TNode(5);
        preorder(root); System.out.println(); // 1 2 4 5 3
        inorder(root); System.out.println(); // 4 2 5 1 3
        postorder(root); System.out.println(); // 4 5 2 3 1
    }
}
```

Example 2: Iterative inorder with stack

```
import java.util.*;
public class Traversal2 {
    static void inorderIter(TNode root) {
        Stack<TNode> st = new Stack<>();
        TNode cur = root;
        while (cur != null || !st.isEmpty()) {
            while (cur != null) { st.push(cur); cur = cur.left; }
            cur = st.pop();
            System.out.print(cur.data + " ");
            cur = cur.right;
        }
    }
    public static void main(String[] args) {
        TNode root = new TNode(1);
        root.left = new TNode(2); root.right = new TNode(3);
        inorderIter(root); // 2 1 3
    }
}
```

12. Level Order Traversal and Height of Binary Tree

Level-order (BFS) uses a queue. Height = longest root-to-leaf path (edges or nodes; be consistent). Empty tree height = -1 or 0 depending on convention; single node = 0 (edges).

Example 1: Level-order traversal

```
import java.util.*;
public class LevelOrder1 {
    static void levelOrder(TNode root) {
        if (root == null) return;
        Queue<TNode> q = new LinkedList<>();
        q.offer(root);
        while (!q.isEmpty()) {
            TNode n = q.poll();
            System.out.print(n.data + " ");
            if (n.left != null) q.offer(n.left);
            if (n.right != null) q.offer(n.right);
        }
    }
    public static void main(String[] args) {
        TNode root = new TNode(1);
        root.left = new TNode(2); root.right = new TNode(3);
        root.left.left = new TNode(4);
        levelOrder(root); // 1 2 3 4
    }
}
```

Example 2: Height of tree

```
public class LevelOrder2 {
    static int height(TNode r) {
        if (r == null) return -1; // edges convention
        return 1 + Math.max(height(r.left), height(r.right));
    }
    public static void main(String[] args) {
        TNode root = new TNode(1);
        root.left = new TNode(2); root.left.left = new TNode(3);
        System.out.println(height(root)); // 2
    }
}
```

13. BST Operations: Search, Insert and Delete

Binary Search Tree: left subtree keys < node < right subtree keys. Average $O(\log n)$ search/insert/delete; worst $O(n)$ when skewed. Delete has 3 cases: leaf, one child, two children (replace with inorder successor/predecessor).

Example 1: Search & Insert

```
public class BST1 {
    static TNode insert(TNode root, int key) {
        if (root == null) return new TNode(key);
        if (key < root.data) root.left = insert(root.left, key);
        else if (key > root.data) root.right = insert(root.right, key);
        return root;
    }
    static boolean search(TNode root, int key) {
        if (root == null) return false;
        if (root.data == key) return true;
        return key < root.data ? search(root.left, key) : search(root.right, key);
    }
    public static void main(String[] args) {
        TNode root = null;
        root = insert(root, 50); root = insert(root, 30);
        root = insert(root, 70); root = insert(root, 20);
        System.out.println(search(root, 30)); // true
        System.out.println(search(root, 40)); // false
    }
}
```

Example 2: Delete (simplified – leaf & one child shown fully)

```
public class BST2 {
    static TNode minValueNode(TNode n) {
        while (n.left != null) n = n.left;
        return n;
    }
    static TNode delete(TNode root, int key) {
        if (root == null) return null;
        if (key < root.data) root.left = delete(root.left, key);
        else if (key > root.data) root.right = delete(root.right, key);
        else {
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;
            TNode succ = minValueNode(root.right);
            root.data = succ.data;
            root.right = delete(root.right, succ.data);
        }
        return root;
    }
}
```

14. BST Applications and Validation Problems

Applications: ordered maps/sets, symbol tables, range queries, ceiling/floor. Common problems: validate BST, kth smallest, lowest common ancestor, convert sorted array to BST.

Example 1: Validate BST

```
public class BSTApp1 {
    static boolean isValid(TNode root, long min, long max) {
        if (root == null) return true;
        if (root.data <= min || root.data >= max) return false;
        return isValid(root.left, min, root.data) &&
            isValid(root.right, root.data, max);
    }
    public static void main(String[] args) {
        TNode root = new TNode(2);
        root.left = new TNode(1); root.right = new TNode(3);
        System.out.println(isValid(root, Long.MIN_VALUE, Long.MAX_VALUE)); // true
    }
}
```

Example 2: Kth smallest in BST (inorder)

```
public class BSTApp2 {
    static int count = 0, answer = -1;
    static void kthSmallest(TNode root, int k) {
        if (root == null || answer != -1) return;
        kthSmallest(root.left, k);
        count++;
        if (count == k) { answer = root.data; return; }
        kthSmallest(root.right, k);
    }
    public static void main(String[] args) {
        TNode root = new TNode(3);
        root.left = new TNode(1); root.right = new TNode(4);
        root.left.right = new TNode(2);
        kthSmallest(root, 2);
        System.out.println(answer); // 2
    }
}
```

15. Min Heap and Max Heap

Heap = complete binary tree stored in array. Min-heap: parent $\leq$ children; Max-heap: parent $\geq$ children. Index math: parent $(i-1)/2$, left $2i+1$, right $2i+2$. Insert $O(\log n)$, extract-min/max $O(\log n)$, peek $O(1)$.

Example 1: Min-heap insert (bubble up)

```
import java.util.*;
public class Heap1 {
    static void bubbleUp(List<Integer> h, int i) {
        while (i > 0) {
            int p = (i - 1) / 2;
            if (h.get(p) <= h.get(i)) break;
            Collections.swap(h, p, i);
            i = p;
        }
    }
    static void insert(List<Integer> h, int val) {
        h.add(val);
        bubbleUp(h, h.size() - 1);
    }
    public static void main(String[] args) {
        List<Integer> h = new ArrayList<>();
        insert(h, 30); insert(h, 10); insert(h, 20); insert(h, 5);
        System.out.println(h); // min at index 0 → 5
    }
}
```

Example 2: Using PriorityQueue as min/max heap

```
import java.util.*;
public class Heap2 {
    public static void main(String[] args) {
        PriorityQueue<Integer> minH = new PriorityQueue<>();
        PriorityQueue<Integer> maxH = new PriorityQueue<>(Collections.reverseOrder());
        for (int x : new int[]{4, 1, 7, 3}) { minH.offer(x); maxH.offer(x); }
        System.out.println(minH.poll()); // 1
        System.out.println(maxH.poll()); // 7
    }
}
```

16. Heap Sort and Heapify

Heapify (sift-down) restores heap property in $O(\log n)$. Build-heap from array is $O(n)$. Heap Sort: build max-heap then repeatedly extract max $\rightarrow O(n \log n)$, in-place, not stable.

Example 1: Heapify (max-heap sift-down)

```
public class HeapSort1 {
    static void heapify(int[] a, int n, int i) {
        int largest = i, l = 2*i+1, r = 2*i+2;
        if (l < n && a[l] > a[largest]) largest = l;
        if (r < n && a[r] > a[largest]) largest = r;
        if (largest != i) {
            int t = a[i]; a[i] = a[largest]; a[largest] = t;
            heapify(a, n, largest);
        }
    }
    public static void main(String[] args) {
        int[] a = {4, 10, 3, 5, 1};
        for (int i = a.length/2 - 1; i >= 0; i--) heapify(a, a.length, i);
        for (int x : a) System.out.print(x + " "); // max-heap order
    }
}
```

Example 2: Full Heap Sort

```
public class HeapSort2 {
    static void heapSort(int[] a) {
        int n = a.length;
        for (int i = n/2 - 1; i >= 0; i--) heapify(a, n, i);
        for (int i = n - 1; i > 0; i--) {
            int t = a[0]; a[0] = a[i]; a[i] = t;
            heapify(a, i, 0);
        }
    }
    static void heapify(int[] a, int n, int i) { /* same as above */
        int largest = i, l = 2*i+1, r = 2*i+2;
        if (l < n && a[l] > a[largest]) largest = l;
        if (r < n && a[r] > a[largest]) largest = r;
        if (largest != i) {
            int t = a[i]; a[i] = a[largest]; a[largest] = t;
            heapify(a, n, largest);
        }
    }
    public static void main(String[] args) {
        int[] a = {12, 11, 13, 5, 6, 7};
        heapSort(a);
        for (int x : a) System.out.print(x + " "); // 5 6 7 11 12 13
    }
}
```

17. Heap Operations and Priority Queue Applications

Applications: Dijkstra's / Prim's algorithms, Huffman coding, scheduling, median maintenance (two heaps), top-K elements, merge K sorted lists.

Example 1: K largest elements

```
import java.util.*;
public class HeapApp1 {
    static int[] kLargest(int[] a, int k) {
        PriorityQueue<Integer> minH = new PriorityQueue<>();
        for (int x : a) {
            minH.offer(x);
            if (minH.size() > k) minH.poll();
        }
        int[] res = new int[k];
        for (int i = k-1; i >= 0; i--) res[i] = minH.poll();
        return res;
    }
    public static void main(String[] args) {
        int[] r = kLargest(new int[]{3, 1, 5, 12, 2, 11}, 3);
        System.out.println(Arrays.toString(r)); // [5, 11, 12]
    }
}
```

Example 2: Merge K sorted arrays (concept with PQ)

```
import java.util.*;
public class HeapApp2 {
    static class Item {
        int val, arrIdx, elemIdx;
        Item(int v, int a, int e) { val=v; arrIdx=a; elemIdx=e; }
    }
    static List<Integer> mergeK(int[][] arrays) {
        PriorityQueue<Item> pq = new PriorityQueue<>((x,y) -> x.val - y.val);
        List<Integer> res = new ArrayList<>();
        for (int i = 0; i < arrays.length; i++)
            if (arrays[i].length > 0) pq.offer(new Item(arrays[i][0], i, 0));
        while (!pq.isEmpty()) {
            Item cur = pq.poll();
            res.add(cur.val);
            int next = cur.elemIdx + 1;
            if (next < arrays[cur.arrIdx].length)
                pq.offer(new Item(arrays[cur.arrIdx][next], cur.arrIdx, next));
        }
        return res;
    }
}
```

18. AVL Trees and Rotations

AVL = self-balancing BST. Balance factor = $\text{height}(\text{left}) - \text{height}(\text{right}) \in \{-1, 0, 1\}$. Four rotations: LL (right rotate), RR (left rotate), LR (left then right), RL (right then left). Guarantees $O(\log n)$ operations.

Example 1: Right rotation (LL case)

```
class AVLNode {
    int data, height;
    AVLNode left, right;
    AVLNode(int d) { data = d; height = 1; }
}

public class AVL1 {
    static int height(AVLNode n) { return n == null ? 0 : n.height; }
    static AVLNode rightRotate(AVLNode y) {
        AVLNode x = y.left;
        AVLNode T2 = x.right;
        x.right = y; y.left = T2;
        y.height = 1 + Math.max(height(y.left), height(y.right));
        x.height = 1 + Math.max(height(x.left), height(x.right));
        return x;
    }
    public static void main(String[] args) {
        // Conceptual: after inserting 30,20,10 → LL imbalance → rightRotate
        System.out.println("Right rotation restores LL imbalance");
    }
}
```

Example 2: Balance factor helper

```
public class AVL2 {
    static int balanceFactor(AVLNode n) {
        return n == null ? 0 : height(n.left) - height(n.right);
    }
    static int height(AVLNode n) { return n == null ? 0 : n.height; }
    // Insert: normal BST insert, update heights, check BF,
    // apply LL/RR/LR/RL rotation as needed.
    public static void main(String[] args) {
        System.out.println("BF in {-1,0,1} for every node ⇒ AVL balanced");
    }
}
```

19. Red-Black Trees and Self-Balancing Concepts

Red-Black Tree = BST with colour (red/black) constraints that keep the tree approximately balanced. Properties: (1) node is red or black (2) root is black (3) all leaves (NIL) are black (4) red node has black children (5) every path from node to descendant leaves has same # of black nodes. Used in Java TreeMap / TreeSet. Slightly more relaxed than AVL → fewer rotations on insert/delete.

Example 1: Colour properties illustration (conceptual code)

```
enum Color { RED, BLACK }
class RBNode {
    int data; Color color;
    RBNode left, right, parent;
    RBNode(int d) { data = d; color = Color.RED; } // new nodes are red
}
public class RBTree1 {
    public static void main(String[] args) {
        // Insert may require recolouring + rotations
        // to restore the 5 Red-Black properties.
        System.out.println("Java TreeMap is a Red-Black Tree");
    }
}
```

Example 2: Why self-balancing matters

```
import java.util.*;
public class RBTree2 {
    public static void main(String[] args) {
        // Sorted inserts into a plain BST → O(n) skewed tree
        // TreeMap (RB-tree) stays O(log n)
        TreeMap<Integer, String> map = new TreeMap<>();
        for (int i = 1; i <= 5; i++) map.put(i, "v" + i);
        System.out.println(map.firstKey()); // 1
        System.out.println(map.higherKey(2)); // 3
        // ceiling, floor, subMap all O(log n)
    }
}
```

You now have complete coverage of Day-4 DSA topics with explanations and two working programs each. Practice drawing trees/heaps and dry-running rotations and pointer updates on paper.

[All the best for your placements!](#)